

# Lab 06 - Labels, Selectors and ReplicaSets

|             |                                                                                                                                                                                                                |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lab id      | Lab 06                                                                                                                                                                                                         |
| Day / Topic | Day 2 - Workloads & Scheduling                                                                                                                                                                                 |
| Duration    | 45 minutes                                                                                                                                                                                                     |
| Namespace   | kcna-lab06                                                                                                                                                                                                     |
| Mapping     | LO2 *Identify the technical and practical requirements in a Kubernetes setup*   A3 *Identify technical and practical requirements as well as stakeholders' demands*   K3 *Objectives of solution architecture* |
| Cluster     | Single-node kind cluster, Kubernetes v1.30+                                                                                                                                                                    |

---

## 1. Objective

By the end of this lab you will be able to:

1. Apply the Marina Freight Systems label standard to Kubernetes objects and query the cluster with both equality-based (`-l app=tracking-api`) and set-based (`-l 'app in (a,b)'`) selectors.
2. Explain that a ReplicaSet has no pointer to its Pods - the only binding is `.spec.selector` matching Pod labels.
3. Distinguish `matchLabels` from `matchExpressions` and use the `In`, `NotIn` and `Exists` operators.
4. Demonstrate adoption (a controller takes ownership of a pre-existing Pod) and orphaning (a relabelled Pod is released and replaced).
5. Diagnose the API-server rejection that occurs when a selector does not match its own Pod template.

---

## 2. Prerequisites

A running single-node kind cluster and a `kubectl` that can reach it.

```
kubectl version --output=yaml | grep -E 'gitVersion|major|minor' | head -6
```

Expected output (your patch version will differ; anything v1.30 or newer is fine):

```
major: "1"
minor: "31"
gitVersion: v1.31.0
major: "1"
minor: "31"
gitVersion: v1.31.0
```

Confirm you have exactly one node and that it is Ready:

```
kubectl get nodes
```

Expected output:

| NAME               | STATUS | ROLES         | AGE | VERSION |
|--------------------|--------|---------------|-----|---------|
| kind-control-plane | Ready  | control-plane | 21m | v1.31.0 |

Change into the lab directory - every path in this lab is relative to it:

```
cd courseware/labs/lab-06-labels-selectors-replicasets
ls
```

Expected output:

```
README.md  brief.json  data  manifests  verification
```

---

### 3. Scenario

Marina Freight Systems (MFS) is a Singapore port-logistics SaaS provider. Its platform team runs the \*Harbour\* platform on Kubernetes and is preparing release r24.

An incident review found that a routine `kubectl delete pod -l app=tracking` command took down two unrelated services, because nobody could say with confidence which Pods a given label selector actually matched. The platform lead has published a label standard in `data/service-inventory.csv` and asked you to:

- \* load the inventory into the cluster so it is queryable,
- \* stand up the tracking-api and berth-status workloads under the standard,
- \* and prove - with evidence, not assertion - exactly which Pods each selector binds.

There is one complication. A Pod named `tracking-api-legacy` was created by hand during a previous incident and is still serving traffic. Management will not allow it to be deleted. You must show what happens when a ReplicaSet is introduced whose selector already matches it.

---

### 4. Step-by-step procedure

#### Step 1 - Create the namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab06 created
```

Confirm it, and note the labels the manifest set:

```
kubectl get ns kcna-lab06 --show-labels
```

Expected output:

| NAME       | STATUS | AGE | LABELS                                                                              |
|------------|--------|-----|-------------------------------------------------------------------------------------|
| kcna-lab06 | Active | 4s  | course=kcna, kubernetes.io/metadata.name=kcna-lab06, lab=lab-06, owner=mfs-platform |

kubernetes.io/metadata.name is added automatically by the API server on every namespace. You will use it in Lab 16 to write NetworkPolicies that select namespaces.

## Step 2 - Load the MFS service inventory into the cluster

Look at the dataset first. This CSV is the source of truth for every label value you apply in this lab.

```
cat data/service-inventory.csv
```

Expected output:

```
service_name,app,tier,env,release,owner_team,replicas
Container Tracking API,tracking-api,edge,staging,r24,mfs-platform,3
Berth Status Service,berth-status,internal,staging,r24,mfs-platform,2
Cargo Manifest Store,cargo-manifest,internal,staging,r24,mfs-data,2
Customs Declaration Gateway,customs-gateway,edge,staging,r24,mfs-compliance,3
Tariff Rate Cache,tariff-cache,internal,staging,r24,mfs-data,2
Vessel ETA Predictor,vessel-eta,experimental,staging,r23,mfs-labs,1
Yard Crane Telemetry,crane-telemetry,internal,staging,r24,mfs-ops,2
Driver Mobile Backend,driver-backend,edge,staging,r24,mfs-platform,3
Warehouse Slotting Engine,slotting-engine,internal,staging,r23,mfs-ops,2
Partner Webhook Relay,webhook-relay,experimental,staging,r23,mfs-labs,1
```

Publish it as a ConfigMap so workloads in the cluster can read it:

```
kubectl -n kcna-lab06 create configmap service-inventory \
  --from-file=data/service-inventory.csv
```

Expected output:

```
configmap/service-inventory created
```

Verify the key name - it is the file's base name, which is what the audit Pod mounts:

```
kubectl -n kcna-lab06 get configmap service-inventory -o jsonpath='{.data}' | head -c 120; echo
```

Expected output:

```
{"service-inventory.csv":"service_name,app,tier,env,release,owner_team,replicas\nContainer
Tracking API,tracki
```

## Step 3 - Run the data-driven label audit

```
kubectl apply -f manifests/40-pod-label-audit.yaml
```

Expected output:

```
pod/label-audit created
```

Wait for it to finish (it runs once and exits):

```
kubectl -n kcna-lab06 wait --for=jsonpath='{.status.phase}'=Succeeded pod/label-audit --
timeout=90s
```

Expected output:

```
pod/label-audit condition met
```

Read what it computed from the CSV:

```
kubectl -n kcna-lab06 logs label-audit
```

Expected output:

```
== MFS service inventory: tier=edge services ==
Container Tracking API app=tracking-api tier=edge env=staging release=r24
Customs Declaration Gateway app=customs-gateway tier=edge env=staging release=r24
Driver Mobile Backend app=driver-backend tier=edge env=staging release=r24
== counts by tier ==
edge=3
experimental=2
internal=5
AUDIT-COMPLETE
```

Three edge services, two experimental, five internal - ten in total. tracking-api is an edge service on release r24. That is the label set you will use next; it is not invented, it comes from the inventory.

### Step 4 - Create the hand-made Pod (the "legacy" workload)

This Pod carries app=tracking-api,tier=edge - precisely what the ReplicaSet in Step 5 will select on.

```
kubectl apply -f manifests/10-pod-adoption-candidate.yaml
```

Expected output:

```
pod/tracking-api-legacy created
```

```
kubectl -n kcna-lab06 get pod tracking-api-legacy --show-labels
```

Expected output:

| NAME                | READY | STATUS  | RESTARTS | AGE | LABELS                                                                     |
|---------------------|-------|---------|----------|-----|----------------------------------------------------------------------------|
| tracking-api-legacy | 1/1   | Running | 0        | 12s | app=tracking-api,env=staging,provenance=hand-created,release=r24,tier=edge |

Confirm it has no owner - nothing in the cluster manages it:

```
kubectl -n kcna-lab06 get pod tracking-api-legacy \
-o jsonpath='{.metadata.ownerReferences}'; echo
```

Expected output (an empty line - the field is absent):

### Step 5 - Create the tracking-api ReplicaSet and watch adoption

Read the selector before you apply it:

```
grep -A5 'selector:' manifests/20-replicaset-tracking-api.yaml
```

Expected output:

```
selector:
  matchLabels:
    app: tracking-api
    tier: edge
template:
```

It asks for 3 replicas. There is already 1 matching Pod. Predict the outcome, then apply:

```
kubectl apply -f manifests/20-replicaset-tracking-api.yaml
```

Expected output:

```
replicaset.apps/tracking-api created
```

```
kubectl -n kcna-lab06 get rs tracking-api
```

Expected output:

| NAME         | DESIRED | CURRENT | READY | AGE |
|--------------|---------|---------|-------|-----|
| tracking-api | 3       | 3       | 3     | 15s |

Now the key observation - list the Pods with their provenance label:

```
kubectl -n kcna-lab06 get pods -l app=tracking-api \
-L provenance,tier,release
```

Expected output (the two generated Pod name suffixes will differ):

| NAME                | READY | STATUS  | RESTARTS | AGE | PROVENANCE   | TIER | RELEASE |
|---------------------|-------|---------|----------|-----|--------------|------|---------|
| tracking-api-4m2ql  | 1/1   | Running | 0        | 20s | replicaset   | edge | r24     |
| tracking-api-legacy | 1/1   | Running | 0        | 75s | hand-created | edge | r24     |
| tracking-api-x7ntb  | 1/1   | Running | 0        | 20s | replicaset   | edge | r24     |

Only two new Pods were created, not three. The ReplicaSet counted the pre-existing Pod towards its replica goal. Prove ownership changed:

```
kubectl -n kcna-lab06 get pod tracking-api-legacy \
-o jsonpath='{.metadata.ownerReferences[0].kind}/{.metadata.ownerReferences[0].name}
controller={.metadata.ownerReferences[0].controller}'; echo
```

Expected output:

```
ReplicaSet/tracking-api controller=true
```

This is adoption: the ReplicaSet controller wrote an ownerReference into a Pod it did not create, purely because the labels matched.

## Step 6 - Orphan a Pod and watch the replacement appear

Open a watch in a second terminal:

```
kubectl -n kcna-lab06 get pods -l app=tracking-api -w
```

In your first terminal, relabel the legacy Pod out of the selector - this is the standard "quarantine a Pod for debugging" technique:

```
kubectl -n kcna-lab06 label pod tracking-api-legacy tier=quarantine --overwrite
```

Expected output:

```
pod/tracking-api-legacy labeled
```

The watch terminal shows a new Pod being created within a second or two:

```
tracking-api-9kdd8    0/1    Pending           0      0s
tracking-api-9kdd8    0/1    ContainerCreating  0      0s
tracking-api-9kdd8    1/1    Running           0      2s
```

Stop the watch with Ctrl-C. Now confirm the legacy Pod was released, not deleted:

```
kubectl -n kcna-lab06 get pod tracking-api-legacy \
-o jsonpath='{.metadata.ownerReferences}'; echo
```

Expected output (empty again - the ownerReference was removed):

```
kubectl -n kcna-lab06 get pods --show-labels
```

Expected output:

| NAME                | READY | STATUS    | RESTARTS | AGE | LABELS                                                                           |
|---------------------|-------|-----------|----------|-----|----------------------------------------------------------------------------------|
| label-audit         | 0/1   | Completed | 0        | 5m  | app=label-audit,env=staging,release=r24,tier=tooling                             |
| tracking-api-4m2ql  | 1/1   | Running   | 0        | 3m  | app=tracking-api,env=staging,provenance=replicaset,release=r24,tier=edge         |
| tracking-api-9kdd8  | 1/1   | Running   | 0        | 40s | app=tracking-api,env=staging,provenance=replicaset,release=r24,tier=edge         |
| tracking-api-legacy | 1/1   | Running   | 0        | 4m  | app=tracking-api,env=staging,provenance=hand-created,release=r24,tier=quarantine |
| tracking-api-x7ntb  | 1/1   | Running   | 0        | 3m  | app=tracking-api,env=staging,provenance=replicaset,release=r24,tier=edge         |

The quarantined Pod is still Running and still serving - it is simply no longer counted. This is the safest way to pull a misbehaving Pod out of a Service for inspection.

Restore it so the ReplicaSet re-adopts it and scales itself back down:

```
kubectl -n kcna-lab06 label pod tracking-api-legacy tier=edge --overwrite
sleep 5
kubectl -n kcna-lab06 get pods -l app=tracking-api
```

Expected output - the ReplicaSet now owns 4 Pods for a desired count of 3, so it deletes the newest one and settles back to 3:

| NAME                | READY | STATUS  | RESTARTS | AGE |
|---------------------|-------|---------|----------|-----|
| tracking-api-4m2ql  | 1/1   | Running | 0        | 4m  |
| tracking-api-legacy | 1/1   | Running | 0        | 5m  |
| tracking-api-x7ntb  | 1/1   | Running | 0        | 4m  |

## Step 7 - Set-based selectors with matchExpressions

```
kubectl apply -f manifests/30-replicaset-berth-status.yaml
```

Expected output:

```
replicaset.apps/berth-status created
```

```
kubectl -n kcna-lab06 get rs
```

Expected output:

| NAME         | DESIRED | CURRENT | READY | AGE |
|--------------|---------|---------|-------|-----|
| berth-status | 2       | 2       | 2     | 18s |
| tracking-api | 3       | 3       | 3     | 6m  |

Inspect the compiled selector - kubectl renders set-based expressions in the same syntax you type on the command line:

```
kubectl -n kcna-lab06 get rs berth-status -o jsonpath='{.spec.selector}' | tr ',' '\n'
```

Expected output:

```
{
  "matchExpressions": [
    {
      "key": "tier",
      "operator": "NotIn",
      "values": [
        "experimental"
      ]
    },
    {
      "key": "release",
      "operator": "Exists"
    },
    {
      "key": "app",
      "operator": "In",
      "values": [
        "berth-status"
      ]
    }
  ],
  "matchLabels": {
    "app": "berth-status"
  }
}
```

Now run the equivalent queries yourself. Equality-based:

```
kubectl -n kcna-lab06 get pods -l app=berth-status
```

Expected output:

| NAME               | READY | STATUS  | RESTARTS | AGE |
|--------------------|-------|---------|----------|-----|
| berth-status-hn6bq | 1/1   | Running | 0        | 65s |
| berth-status-tzc4k | 1/1   | Running | 0        | 65s |

Set-based, spanning both workloads and excluding the experimental tier:

```
kubectl -n kcna-lab06 get pods \
  -l 'app in (tracking-api,berth-status),tier notin (experimental)'
```

Expected output:

| NAME                | READY | STATUS  | RESTARTS | AGE |
|---------------------|-------|---------|----------|-----|
| berth-status-hn6bq  | 1/1   | Running | 0        | 2m  |
| berth-status-tzc4k  | 1/1   | Running | 0        | 2m  |
| tracking-api-4m2ql  | 1/1   | Running | 0        | 8m  |
| tracking-api-legacy | 1/1   | Running | 0        | 9m  |
| tracking-api-x7ntb  | 1/1   | Running | 0        | 8m  |

The Exists form, and its negation:

```
kubectl -n kcna-lab06 get pods -l 'release' --no-headers | wc -l
kubectl -n kcna-lab06 get pods -l '!provenance' --no-headers | wc -l
```

Expected output:

```
6
1
```

Six Pods carry a release key of any value; exactly one Pod (label-audit) has no provenance key at all.

## Step 8 - Cross-check the selector against the inventory

Everything in this lab traces back to the CSV. Confirm the count you queried matches what the dataset declares:

```
awk -F, 'NR>1 && $2=="tracking-api" {print "inventory says replicas="$7}' data/service-
inventory.csv
kubectl -n kcna-lab06 get rs tracking-api -o jsonpath='{cluster says replicas={ .spec.replicas }}';
echo
```

Expected output:

```
inventory says replicas=3
cluster says replicas=3
```

---

## 5. Verification

Run the bundled check script:

```
bash verification/checks.sh
```

The full expected transcript is in [`verification/expected-output.md`](#). The script is read-only - it creates and deletes nothing. It exits 0 only when every assertion passes.

---

## 6. Failure injection - a selector that cannot match its own template

manifests/50-replicaset-broken-selector.yaml asks for tier: edge in .spec.selector.matchLabels, but its Pod template stamps tier: internal. Such a ReplicaSet would create Pods it could never own, then create more, forever. The API server refuses it.

```
kubectl apply -f manifests/50-replicaset-broken-selector.yaml
```

Real error text:

The ReplicaSet "cargo-manifest-broken" is invalid: spec.template.metadata.labels: Invalid value: map[string]string{"app":"cargo-manifest", "env":"staging", "release":"r24", "tier":"internal"}: `selector` does not match template `labels`

Diagnosis. Validation for ReplicaSets, Deployments, StatefulSets, DaemonSets and Jobs requires the selector to be a subset of the template labels. Read the message backwards: it prints the *template* labels and tells you the *selector* does not match them. Compare the two blocks directly:

```
grep -A4 'matchLabels:' manifests/50-replicaset-broken-selector.yaml
grep -A5 'template:' manifests/50-replicaset-broken-selector.yaml | grep -E 'app:|tier:'
```

Expected output:

```
matchLabels:
  app: cargo-manifest
  tier: edge
template:
  app: cargo-manifest
  tier: internal
```

tier is edge in the selector and internal in the template. Fix it by making them agree - the inventory says cargo-manifest is an internal service, so the selector is the wrong one:

```
sed 's/      tier: edge/      tier: internal/' manifests/50-replicaset-broken-selector.yaml \
| kubectl apply -f -
```

Expected output:

```
replicaset.apps/cargo-manifest-broken created
```

```
kubectl -n kcna-lab06 get rs cargo-manifest-broken
```

Expected output:

| NAME                  | DESIRED | CURRENT | READY | AGE |
|-----------------------|---------|---------|-------|-----|
| cargo-manifest-broken | 2       | 2       | 2     | 20s |

Second failure to try if time allows. Delete one of the ReplicaSet's Pods (kubectl -n kcna-lab06 delete pod <a tracking-api pod>) and watch a replacement appear in under two seconds. The ReplicaSet is a *reconciliation loop* over a label query\*, not a list of Pod names - that is the whole idea.

---

## 7. Troubleshooting

| Symptom                                                                                                                | Cause                                                                                                | Fix                                                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The ReplicaSet "x" is invalid:<br>spec.template.metadata.labels ...<br>\selector\ does not match template<br>\labels\` | .spec.selector is not a subset of<br>.spec.template.metadata.labels                                  | Make every selector key/value appear<br>identically in the template labels; the<br>template may carry extra labels, the<br>selector may not                                                 |
| ReplicaSet reports DESIRED 3 /<br>CURRENT 5 and keeps deleting Pods                                                    | Another Pod or controller in the same<br>namespace carries labels that also<br>satisfy this selector | Run <code>kubectl -n kcna-lab06 get pods -l<br/>&lt;selector&gt; --show-labels</code> ; make selectors<br>more specific (add app and tier), or<br>relabel the intruder                      |
| You delete a Pod and it "comes back"                                                                                   | The owning ReplicaSet reconciled the<br>label query and recreated it                                 | Delete or scale the ReplicaSet, not the<br>Pod: <code>kubectl -n kcna-lab06 scale<br/>rs/tracking-api --replicas=0</code>                                                                   |
| kubectl label returns error: 'tier' already<br>has a value (edge), and --overwrite is<br>false                         | kubectl label refuses to change an<br>existing key by default                                        | Add --overwrite                                                                                                                                                                             |
| Set-based query returns nothing and no<br>error                                                                        | Shell expanded the parentheses or ! in<br>the selector                                               | Quote the whole selector in single<br>quotes: <code>-l 'app in (a,b)', -l '!provenance'</code>                                                                                              |
| label-audit Pod is<br>CreateContainerConfigError                                                                       | The ConfigMap service-inventory was<br>not created, or was created with a<br>different key name      | <code>kubectl -n kcna-lab06 get cm<br/>service-inventory -o jsonpath='{.data}';<br/>recreate with<br/>--from-file=data/service-inventory.csv so<br/>the key is service-inventory.csv</code> |

---

## 8. Cleanup

Delete only this lab's namespace. Everything you created lives inside it.

```
kubectl delete namespace kcna-lab06
```

Expected output:

```
namespace "kcna-lab06" deleted
```

Confirm:

```
kubectl get ns kcna-lab06
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab06" not found
```

---

## 9. What you learned

\* A label is arbitrary key/value metadata; a selector is a query over it. Kubernetes controllers are built almost entirely out of this one pairing - there are no Pod-name lists anywhere in a ReplicaSet spec.

- \* matchLabels is equality-based and ANDed. matchExpressions adds the set operators In, NotIn, Exists, DoesNotExist; both may be present and are ANDed together.
- \* On the command line, -l k=v is equality-based, -l 'k in (a,b)' / -l 'k notin (a)' is set-based, -l k is Exists and -l '!k' is DoesNotExist.
- \* Adoption: a controller writes an ownerReference into any unowned Pod whose labels satisfy its selector. Orphaning: change the labels and the controller removes the ownerReference and creates a replacement. The Pod itself is untouched - this is how you quarantine a Pod without losing it.
- \* A selector that is not a subset of the template labels is rejected at admission, before any Pod is created.
- \* --show-labels and -L <key> are the two flags that make label debugging tractable.

## 10. Further reading

- \* Labels and Selectors - <<https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/>>
- \* ReplicaSet - <<https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/>>
- \* Owners and Dependents - <<https://kubernetes.io/docs/concepts/overview/working-with-objects/owners-dependents/>>
- \* Recommended Labels - <<https://kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/>>
- \* kubectl label reference - <[https://kubernetes.io/docs/reference/kubectl/generated/kubectl\\_label/](https://kubernetes.io/docs/reference/kubectl/generated/kubectl_label/)>